AI VIET NAM
@aivietnam.edu.vn

Quiz: Transformation with dbt

Lê Thanh Bảo Trân **Huỳnh Trần Anh Thư**

Proofreading:

Hướng dẫn

Quiz gồm 11 câu trắc nghiệm, bám theo ba phần đầu của bài *Transformation with dbt*: vì sao có dbt, dbt hoạt động thế nào và thiết kế dữ liệu. Mỗi câu chọn một đáp án đúng. Đáp án và giải thích ở cuối tài liệu.

Phần I–II. dbt là gì, hoạt động thế nào

Câu 1. Vì sao các tổ chức chuyển từ ETL sang ELT?

a) SQL chạy nhanh hơn Python.

b) Lưu trữ và tính toán trên cloud warehouse đã rẻ và mạnh, nên có thể nạp dữ liệu thô vào trước rồi biến đổi ngay trong warehouse.

c) ETL không xử lý được dữ liệu dạng CSV.

d) dbt bắt buộc phải dùng ELT.

Câu 2. Phát biểu nào sau đây *sai* về dbt?

a) dbt biên dịch SQL kết hợp Jinja thành SQL thuần.

b) dbt tự xử lý dữ liệu bằng engine riêng của nó.

c) dbt điều phối thứ tự chạy model theo DAG.

d) dbt không thực hiện bước extract và load.

Câu 3. Một model trong dbt là gì?

a) Một file `.sql` chứa đúng một câu `SELECT`; dbt tự bọc câu này thành lệnh tạo view hoặc table.

b) Một file YAML khai báo tên bảng và các cột.

c) Một file `.sql` chứa lệnh `CREATE TABLE` và `INSERT`.

d) Một bảng được tạo thủ công trong warehouse.

Câu 4. Phát biểu nào đúng về `source()` và `ref()`?

a) `source()` trỏ tới một model khác, còn `ref()` trỏ tới dữ liệu thô.

b) Cả hai chỉ dùng để viết tắt tên bảng, không ảnh hưởng tới thứ tự chạy model.

c) `source()` trỏ tới dữ liệu thô khai báo trong YAML và là điểm bắt đầu của DAG; `ref()` trỏ tới model khác và ngầm khai báo phụ thuộc.

d) `ref()` chỉ được dùng ở tầng staging.

Câu 5. Cho bốn materialization và bốn mô tả. Cách ghép nào đúng?

A.	view	**1.**	Không tạo object, được chèn vào model khác dưới dạng CTE.
B.	table	**2.**	Chỉ lưu câu query, mỗi lần đọc thì tính lại.
C.	incremental	**3.**	Lần đầu tạo bảng, các lần sau chỉ xử lý dòng mới hoặc dòng thay đổi.
D.	ephemeral	**4.**	Chạy query, lưu kết quả và build lại toàn bộ mỗi lần.

a) A–4, B–2, C–3, D–1

b) A–2, B–4, C–1, D–3

c) A–1, B–4, C–3, D–2

d) A–2, B–4, C–3, D–1

Câu 6. Một model incremental lọc theo điều kiện `updated_at > max(updated_at)` và không khai báo `unique_key`. Cách sửa nào đúng và đủ?

a) Lọc lùi một khoảng an toàn để lấy cả dữ liệu đến trễ, đồng thời khai báo `unique_key` để các dòng lấy lại được ghi đè thay vì chèn trùng.

b) Chỉ cần khai báo `unique_key`.

c) Chỉ cần lọc lùi 3 ngày.

d) Đổi materialization sang `view`.

Phần III. Thiết kế dữ liệu

Câu 7. Việc nào *không* nên làm ở tầng staging?

a) Đổi tên cột, ví dụ `id` thành `order_id`.

b) Ép kiểu dữ liệu, ví dụ chuỗi thành `date`.

c) Join với bảng khác hoặc dùng `group by`.

d) Chuẩn hoá giá trị bằng `lower()`, `trim()`.

Câu 8. Trường hợp nào *không* cần tạo model intermediate?

a) Logic quá dài làm model mart khó đọc.

b) Cùng một logic được dùng lại ở nhiều mart.

c) Cần đổi grain trước khi join.

d) Mart chỉ cần join hai model staging đơn giản.

Câu 9. Phát biểu nào đúng về fact table và dimension table trong star schema?

a) Fact ghi lại sự kiện đo đếm được, chứa chỉ số và khoá ngoại; dimension mô tả ngữ cảnh (ai, cái gì, khi nào), dùng để lọc và nhóm.

b) Fact mô tả ngữ cảnh, dimension chứa các chỉ số như số lượng, số tiền.

c) Fact và dimension bắt buộc phải có cùng grain.

d) Dimension là nơi chứa các cột tiền để cộng tổng doanh thu.

Câu 10. Fan-out xảy ra khi nào?

a) Khi dùng `group by` ở tầng staging.

b) Khi `left join` làm xuất hiện giá trị null.

c) Khi model được materialize dạng view thay vì table.

d) Khi join hai quan hệ một-nhiều theo cùng một khoá, làm số dòng bị nhân lên.

Câu 11. Một số đơn hàng chưa có lần thanh toán nào. Sau khi gom `payments` về grain đơn hàng và `left join` vào `orders`, cột `total_paid` của các đơn này bị null. Nên xử lý dữ liệu thiếu này ở đâu và như thế nào?

a) Ở staging: lọc bỏ các đơn chưa thanh toán khỏi `stg_orders`.

b) Ở bước join (intermediate hoặc mart): dùng `coalesce(total_paid, 0)`.

c) Giữ nguyên null, để dashboard tự xử lý.

d) Đổi `left join` thành `inner join` để không còn dòng null.

Đáp án và giải thích

Câu	1	2	3	4	5	6	7	8	9	10	11
Đáp án	b	b	a	c	d	a	c	d	a	d	b

Câu 1. b. Trong ETL, dữ liệu được biến đổi trên máy chủ trung gian vì lưu trữ và tính toán trong warehouse on-premise đắt. Cloud warehouse và các engine cột như DuckDB làm cho lưu trữ rẻ, tính toán mạnh và tách rời nhau. Dữ liệu thô luôn còn trong kho, nên khi logic sai chỉ cần chạy lại phần biến đổi.

Câu 2. b. dbt không xử lý dữ liệu. Mọi phép tính do warehouse thực hiện trên câu SQL mà dbt sinh ra, nên hiệu năng phụ thuộc vào warehouse.

Câu 3. a. Model là một file `.sql` chứa đúng một câu `SELECT`, tên file là tên bảng hoặc view được tạo. dbt tự bọc câu `SELECT` thành lệnh tạo object phù hợp theo materialization, nên không viết `CREATE TABLE` hay `INSERT`.

Câu 4. c. `source()` trỏ tới dữ liệu thô khai báo trong YAML, là điểm bắt đầu của DAG và cho phép kiểm tra độ tươi bằng `dbt source freshness`. Mỗi lần gọi `ref()` là ngầm khai báo một phụ thuộc; dbt dựa vào đó để dựng DAG và đổi tên model thành tên đầy đủ theo target (dev hoặc prod).

Câu 5. d. View chỉ lưu câu query (A–2). Table lưu kết quả và build lại toàn bộ (B–4). Incremental chỉ xử lý dòng mới hoặc thay đổi sau lần đầu (C–3). Ephemeral không tạo object mà được chèn vào model khác dưới dạng CTE (D–1).

Câu 6. a. Model này có hai lỗi. Dữ liệu đến trễ (phát sinh sớm nhưng nạp muộn) không bao giờ thoả điều kiện lọc nên bị bỏ qua vĩnh viễn; cách phòng là lọc lùi, ví dụ `max(updated_at) - interval 3 day`. Không có `unique_key` thì dbt chỉ thêm dòng, nên đơn đổi trạng thái bị chèn trùng; cách phòng là khai báo `unique_key` để dbt merge. Hai cách phải đi cùng nhau, vì lọc lùi nghĩa là chủ động lấy lại dòng cũ: (b) và (c) mỗi cách chỉ sửa một nửa.

Câu 7. c. Join làm staging phụ thuộc vào bảng khác, còn `group by` làm đổi grain. Staging chỉ làm sạch từng bảng nguồn theo quan hệ 1:1.

Câu 8. d. Nếu mart chỉ cần join hai model staging đơn giản thì làm thẳng ở mart. Intermediate dành cho ba trường hợp còn lại: logic quá dài, logic dùng lại ở nhiều mart, hoặc cần đổi grain trước khi join.

Câu 9. a. Fact ghi lại sự kiện (một lần bán, một lần thanh toán) với chỉ số và khoá ngoại tới dimension. Dimension mô tả ngữ cảnh của sự kiện, gồm các cột dùng để lọc và nhóm. Mỗi bảng có grain riêng của nó.

Câu 10. d. Grain là câu trả lời cho câu hỏi "một dòng trong bảng này đại diện cho cái gì". Khi join hai quan hệ một-nhiều theo cùng khoá, số dòng bị nhân lên và mọi phép `sum()` phía sau bị đội lên. Lỗi khó phát hiện vì câu SQL không báo lỗi, phần lớn bản ghi có hệ số nhân bằng 1 nên tổng chỉ lệch vài phần trăm, và lỗi có thể xuất hiện muộn khi dữ liệu nguồn thay đổi dù model không đổi.

Câu 11. b. Giá trị null xuất hiện do phép `left join`, nên xử lý ngay ở model thực hiện phép join đó: `coalesce(total_paid, 0)` đúng nghĩa "chưa trả đồng nào". (a) sai vì staging không lọc bỏ dòng theo quy tắc nghiệp vụ, và đơn chưa thanh toán vẫn là dữ liệu cần báo cáo. (c) sai vì mỗi dashboard sẽ xử lý một kiểu, quay lại vấn đề hai dashboard hai con số. (d) sai vì `inner join` làm mất luôn các đơn chưa thanh toán. Để phát hiện dữ liệu thiếu ngay từ nguồn, đặt test `not_null` ở staging; để chặn null ở bảng cuối, đặt `not_null` trên cột đó ở mart.